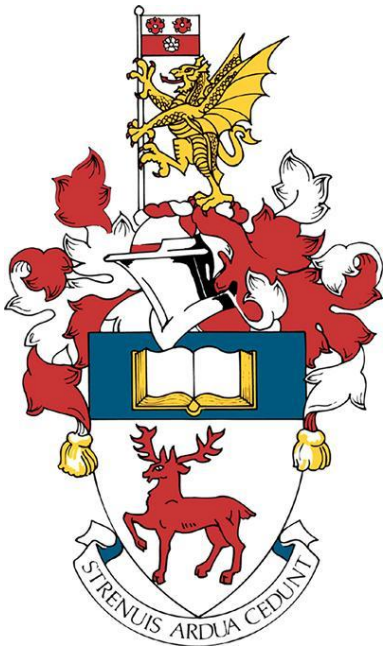
INTRODUCTION TO MACHINE LEARNING (FEEG6042) - ASSIGNMENT



"I confirm that this document is all my own work and that I have not used any Generative AI tools as prohibited in the Tier 1 GenAI guidance. I have read and understood the university's and faculty's academic integrity policies available in section of the university calendar and the programme handbook. I am aware of the requirements of good academic practice and the potential penalties for any breaches."

Student ID: 32948646

Word count: 2725 words

Contents

1. Background

This section presents the theoretical concepts required to investigate overfitting in convolutional neural networks (CNNs). The focus is on how model complexity and training data size influence generalisation performance.

1.1 Supervised Learning and Image Classification

In supervised learning, a model is trained using labelled input–output pairs and learns by adjusting its parameters to minimise a loss function that measures the difference between the predicted and true outputs. In image classification, the inputs are images, and the outputs are class labels, making this a supervised classification problem. The dataset is split into a training set and a test/validation set so that performance on unseen data can be used to assess generalisation and detect overfitting.

1.2 Overfitting

Overfitting occurs when a model fits too closely to the training data and performs very well on the training set but does not generalise well to unseen test data. In that case, the training accuracy becomes very high, while the test accuracy remains lower. At the same time, the training loss continues to decrease while the validation loss stops improving or even begins to increase. Overfitting is most likely to occur when:

- The model is highly complex and contains many trainable parameters
- The training dataset is small (in respect to the complexity of the model)

We need to always have more training data than unknown parameters (θ). As a rule of thumb we need at least 10 times more data than we have unknown parameters, to properly train these models.

1.3 Convolutional Neural Networks (CNNs)

In image classification, the input is a 2-D image rather than a simple vector. CNNs are designed to process image data using convolutional filters and pooling layers. The convolutional layers extract local features from the image, and the pooling layers reduce the size of the feature maps and the number of parameters. CNNs preserve spatial structure and use a lot less parameters than what the Fully Connected Neural Networks would need. For this reason, CNNs are used throughout this assignment rather than FCNNs which are computationally inefficient for image classification tasks.

1.4 Model Complexity and Trainable Parameters

The complexity of a neural network is related to how many trainable parameters it has. Increasing the number of parameters, increases the model's ability to fit the training data. However, as model complexity increases, the risk of overfitting also increases.

A model with a very large number of parameters can achieve very high training accuracy, but this does not guarantee good performance on unseen test data. The training error would become very small while the test error would remain high. This shows that good training performance alone does not imply good generalisation.

1.5 Relationship Between Training Data and Model Size

There is a trade-off between model complexity and training data size. As model complexity increases, a larger amount of training data is required to properly train the model and ensure reliable generalisation. When a complex model is trained on limited data, it is likely to memorise the training set instead of learning general patterns, leading to poor test performance. This trade-off provides the foundation for the experiments.

1.6 Performance Evaluation and Detection of Overfitting

In classification problems, accuracy is the primary performance metric, defined as the proportion of correctly classified samples. In addition, cross-entropy loss measures how different the predicted outputs are from the true class labels. Accuracy only reflects whether the prediction is correct or not, whereas loss also reflects how well the network's output matches the target. For this reason, both accuracy and loss are studied during training.

During training, the network updates its parameters by minimising the loss function, not the accuracy. Accuracy is not used to update the model but is calculated alongside the loss to monitor performance. Loss controls learning, while accuracy is used purely for evaluation.

To detect overfitting, both training and validation performance must be plotted against the number of epochs. Overfitting is identified through:

- A growing gap between training and test accuracy, and
- Divergence between training and validation loss during training.

Although convergence plots provide useful information about the learning behaviour of our model, the final evaluation must always be based on test performance, as this best reflects the model's ability to generalise.

2. Experimental Design

The design follows the workflow: data loading and pre-processing, model selection, defining performance measures, training, and testing. The aim is to demonstrate overfitting in CNNs by controlling the amount of training data and the complexity of the model.

The experiments carried out are outlined below:

1. **Base model:** a CNN model trained on the full MNIST training set, that doesn't overfit
2. **Experiment 1 (reduced data):** the same CNN model trained on a smaller subset of the training set and for more epochs, to demonstrate overfitting
3. **Experiment 2 (increased complexity):** a more complex CNN from the base model with more trainable parameters, trained on the full training set, to demonstrate overfitting

2.1 Data and Pre-processing

The experiments were performed in a supervised learning setting, using the MNIST dataset loaded from Keras: 60,000 training images and 10,000 test images of size 28×28, with labels in $\{0, \dots, 9\}$.

The classification task is simplified to even vs odd digit recognition. The original labels are converted to binary values, so all even digits form *"class 0"* and all odd digits *"class 1"*. The binary labels are then one-hot encoded into 2-dimensional vectors, which matches the two *softmax* output units and allow the use of *categorical cross-entropy* as the loss function. *Categorical cross-entropy* is used as the loss function because the model outputs two softmax probabilities (even and odd), and the target labels are one-hot encoded into two classes.

Before training, the images are converted to *float32* and scaled to ensure that all inputs lie in a consistent numerical range, to train the CNN effectively. Scaling is applied image-by-image rather than feature-by-feature, using the *MinMaxScaler* to map each image's pixel values to the range $[0,1]$. Each image is then reshaped to $(28,28,1)$ so that CNN layers can treat it as a 3-D tensor with a single colour channel (grayscale).

2.2 Base model

The base model is a CNN designed model to perform the even/odd MNIST classification task while avoiding strong overfitting. It follows a standard CNN structure with two convolution–pooling stages followed by a final dense output layer. The convolutional layers use ReLU activation to introduce non-linearity and allow the network to learn more complex features. Whereas the pooling reduces the feature

map size (which reduces computation and helps the network focus on the most important features), and the dense layer uses these features to produce the final classification output.

The first convolutional layer uses 8 filters and the second uses 16 filters, with 2×2 max-pooling applied after each. The network ends with a dense layer with 2 outputs using softmax activation to convert the outputs into class probabilities for even and odd. This relatively small number of filters keeps the number of trainable parameters low, making the model suitable as a non-overfitting reference.

The model is trained using categorical cross-entropy loss and the adam optimiser, with classification accuracy used as the performance metric. Training is performed on the full training set of 60,000 images for 20 epochs with a batch size of 64, while the separate test set of 10,000 images is used for validation.

2.3 Experiment 1 – Reduced Training Data

Experiment 1 investigates overfitting caused by limited training data. The CNN architecture and all training parameters are kept identical to the base model, but only 1,000 training images are used, which is one tenth to those used beforehand. This creates a scenario in which a model of the same complexity is trained on a smaller training dataset. The model will be trained for 100 epochs to show clearly the overfitting nature of the model. The test set remains unchanged, so any decrease in generalisation performance can be attributed directly to the reduced training dataset (and long training). The model is expected to memorise rather than learn from the training data, leading to decreasing training loss while the validation loss increases.

2.4 Experiment 2 – Increased Model Complexity

Experiment 2 investigates overfitting caused by increasing the model complexity while keeping the training data and training procedure the same as in the base model. The architecture is made more complex compared to the base model: the first convolutional layer uses 32 filters and the second uses 64 filters. A third convolutional layer with 64 filters is added, using "same" padding so that the spatial size of the feature maps is preserved at this stage. Each of the first two convolutional layers is still followed by 2×2 max-pooling, as in the base model.

After the convolutional part, the feature maps are flattened and passed through an additional dense hidden layer with 64 units before the final dense layer with 2 outputs and softmax activation. This extra dense layer increases the number of connections and therefore the number of trainable parameters compared to the base model. As in the base model, the loss function, optimiser, and batch size are kept the same. The model is trained on the full training set for 20 epochs, the same as the base CNN. Any change in generalisation performance between the base model and Experiment 2 can

be associated with the increased model complexity (more filters, an extra convolutional layer, and an additional dense layer).

2.5 Training and Evaluation

For all three experiments, training and validation accuracy and loss were recorded at each epoch and plotted as convergence graphs. Overfitting is identified through divergence between training and validation curves. Final model performance is evaluated using the test accuracy and test loss at the end of training. Comparing these results across the three experiments allows a systematic investigation of how training data size and model complexity influence overfitting and generalisation in CNNs.

3. Results

Table 1 below presents all the useful results extracted from the models and the *Figures 1-6* in the next page display the accuracy and loss plots for the base model and the two experiments.

Metric	Base Model	Experiment 1 (Reduced Data)	Experiment 2 (High Complexity)
Training Samples	60,000	1,000	60,000
Number of Epochs	20	100	20
Final Train Accuracy	0.990	1.000	1.000
Final Test Accuracy	0.989	0.950	0.996
Final Train Loss	0.033	0.010	0.002
Final Test Loss	0.038	0.150	0.030
Trainable Parameters	**2,050**	**2,050**	**158,338**
Training Time (s)	46.06	31.28	124.73
Avg Time / Epoch (s)	2.30	1.56	6.24

Table 1: Summary of CNN Training Results for Base Model and both Experiments.

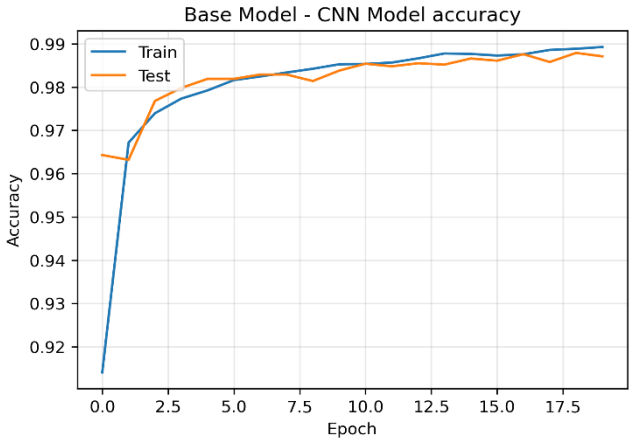


Figure 1: Training and test accuracy for the base CNN model (20 epochs).

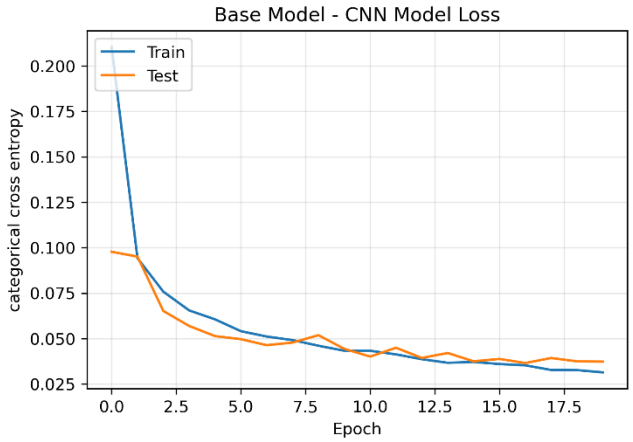


Figure 2: Training and test loss for the base CNN model (20 epochs).

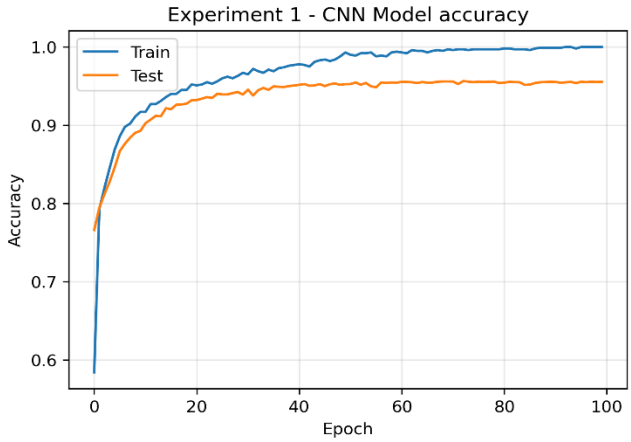


Figure 3: Training and test accuracy for Experiment 1 (reduced training data, 100 epochs).

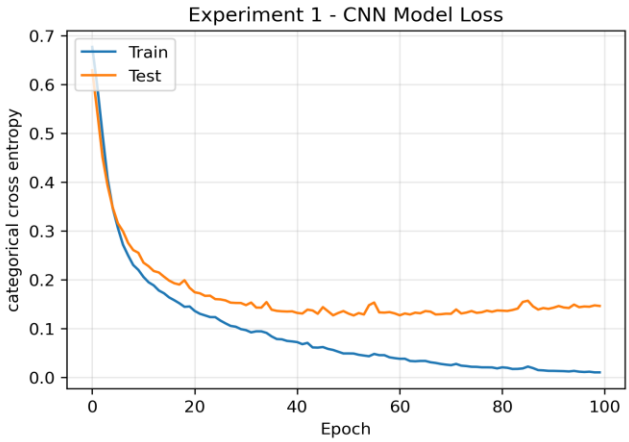


Figure 4: Training and test loss for Experiment 1 (reduced training data, 100 epochs).

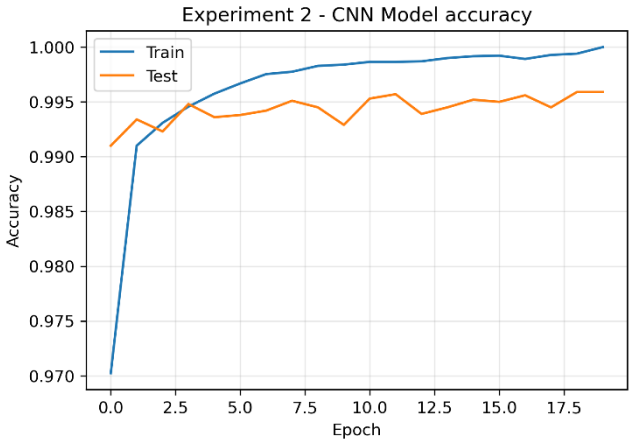


Figure 5: Training and test accuracy for Experiment 2 (high model complexity, 20 epochs).

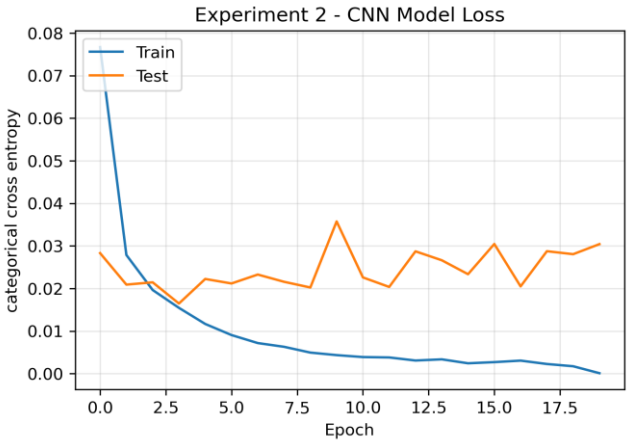


Figure 6: Training and test loss for Experiment 2 (high model complexity, 20 epochs).

4. Discussion

4.1 Base Model

The base model was designed to balance model complexity with the size of the training dataset. As shown in *Figures 1-2*, it achieved a final training accuracy of approximately 99.0% and a test accuracy of approximately 98.9%, with closely matched training and test losses. This indicates that the model fits the data well while maintaining good generalisation and doesn't introduce overfitting. The relatively small number of trainable parameters (2,050) confirms that this performance is achieved with a low-complexity model. The ratio of training data to trainable parameters was ~ 30:1. This is more than the 10 times minimum that we need, so the results are not surprising.

This behaviour is consistent with the theory whereby when the number of trainable parameters is well matched to the available data, the network learns correctly generalisable features rather than memorising individual samples or learning the noise in the data. The relatively small number of parameters in the base CNN limits its capacity to overfit, making it a suitable reference model for the overfitting experiments.

4.2 Experiment 1 – Reduced training dataset

In Experiment 1, the same model trained on only 1,000 images for 100 epochs reaches a training accuracy of 100%, but the test accuracy drops to 95.0%, as shown in *Figure 3*. *Figure 4* shows that the training loss becomes very small, while the test loss increases significantly compared to the base model around the 40th epoch. The model was trained for more than the necessary epochs just to get a better grasp of the performance of the model. The large gap between training and test performance demonstrates overfitting due to insufficient training data. Despite having the same number of parameters as the base model, generalisation performance is significantly worse. In this experiment the ratio of training data to trainable parameters was 2:1. Which is not a good enough ratio as it normally needs to be at least 10 times more.

This behaviour is a great example of overfitting caused by insufficient data. With such a small dataset, the network can easily memorise the training samples instead of learning general features that transfer to unseen data. As a result, the gap between training and test performance becomes large, clearly demonstrating poor generalisation.

4.3 Experiment 2 – High model complexity

In Experiment 2, the more complex CNN trained on the full dataset achieved 100% training accuracy and a test accuracy of 99.6%, as shown inn *Figure 5*. But just because it fits the training data well, it doesn't mean it fits on all data. Trying to exactly match the training data points is wrong because it is like trying to predict the training

set for which you already know what the output is. *Figure 6* illustrates that the training loss drops close to zero while the test loss is unstable and increases with more epochs. This indicates that the model is fitting the training data extremely closely and making generalisation less stable. The number of trainable parameters increases dramatically to 158,338, making the ratio of training data to trainable parameters ~2.6:1. Once again, this is not a good enough ratio, so we expect overfitting.

While the increasing number of parameters allows the model to represent more complex functions, it also increases the risk of overfitting as we don't have an infinite amount of training data to train it with. This experiment confirms that high model capacity alone is enough to induce overfitting, even when a large dataset is available. In this experiment, the complex model has too much flexibility, so instead of learning the true pattern, it starts learning the noise in the data.

4.4 Comparison and Key Findings

Comparing all three models highlights two distinct causes of overfitting. In Experiment 1, overfitting is driven by a low data-to-parameter ratio, leading to memorisation. In Experiment 2, overfitting is driven by excessive model capacity, which allows overly complex representations to be learned.

The base model achieves the best balance between fitting and generalisation, confirming the key principle that good performance depends on matching model complexity to the available data and training duration.

4.5 Training Time and Model Performance

The results also highlight an important trade-off between model complexity, training time, and accuracy. The base model achieves strong test performance with a very small number of trainable parameters and a relatively short training time of 46.06 seconds. This shows that, for this task, a low-complexity CNN is sufficient to extract the relevant features and generalise well to unseen data. From a practical perspective, this makes the base model computationally efficient and well suited for real-world deployment.

In contrast, Experiment 1 has a shorter total training time of 31.28 seconds, despite using more epochs, because it is trained on only 1,000 images. However, this reduced computational cost comes at the expense of significantly worse generalisation, demonstrating that fast training alone does not guarantee good performance. The model memorises the small dataset instead of learning robust features.

Experiment 2 clearly illustrates the opposite extreme. The high-capacity CNN has a dramatically larger number of parameters and requires substantially more computational time per epoch, translating to a total training time of 124.73 seconds. While it achieves very high training accuracy and strong test accuracy, the

improvement over the base model is relatively small when compared to the large increase in computational cost. This confirms the key trade-off of increasing model complexity. It improves representational power, but it also increases training time and the risk of overfitting. Therefore, the best model is not necessarily the most complex one, but the one that provides the optimal balance between accuracy, generalisation, and computational efficiency.

4.6 Improving Generalisation Using Regularisation and Dropout

The overfitting observed in both experiments could be reduced using regularisation methods, which are used to improve generalisation. Weight regularisation adds a penalty to the loss function to limit the size of the trainable parameters, reducing the risk of overfitting, particularly in the high-complexity model of Experiment 2.

Another method is the use of dropout layers, which randomly deactivate a proportion of neurons during training. This prevents the network from becoming too dependent on individual features and helps improve generalisation. Dropout would be beneficial in both the small-data model (Experiment 1) and the high-complexity model (Experiment 2).

These methods are not explored for the purpose of this assignment but are noted next steps

5. Appendix

Training time: 46.06 seconds
Average time per epoch: 2.30 seconds
Model: "sequential_40"

Layer (type)	Output Shape	Param #
conv2d_87 (Conv2D)	(None, 26, 26, 8)	80
max_pooling2d_80 (MaxPooling2D)	(None, 13, 13, 8)	0
conv2d_88 (Conv2D)	(None, 11, 11, 16)	1,168
max_pooling2d_81 (MaxPooling2D)	(None, 5, 5, 16)	0
flatten_40 (Flatten)	(None, 400)	0
dense_47 (Dense)	(None, 2)	802

Total params: 6,152 (24.04 KB)
Trainable params: 2,050 (8.01 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 4,102 (16.03 KB)

Training time: 31.28 seconds
Average time per epoch: 1.56 seconds
Model: "sequential_41"

Layer (type)	Output Shape	Param #
conv2d_89 (Conv2D)	(None, 26, 26, 8)	80
max_pooling2d_82 (MaxPooling2D)	(None, 13, 13, 8)	0
conv2d_90 (Conv2D)	(None, 11, 11, 16)	1,168
max_pooling2d_83 (MaxPooling2D)	(None, 5, 5, 16)	0
flatten_41 (Flatten)	(None, 400)	0
dense_48 (Dense)	(None, 2)	802

Total params: 6,152 (24.04 KB)
Trainable params: 2,050 (8.01 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 4,102 (16.03 KB)

Training time: 124.73 seconds
Average time per epoch: 6.24 seconds
Model: "sequential_42"

Layer (type)	Output Shape	Param #
conv2d_91 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_84 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_92 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_85 (MaxPooling2D)	(None, 5, 5, 64)	0
conv2d_93 (Conv2D)	(None, 3, 3, 64)	36,928
flatten_42 (Flatten)	(None, 1600)	0
dense_49 (Dense)	(None, 64)	102,464
dense_50 (Dense)	(None, 2)	130

Total params: 475,010 (1.81 MB)
Trainable params: 158,338 (618.51 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 316,672 (1.21 MB)

Figure 7: Comparison of CNN Architectures, parameters and training times across all models. Also presented in Table 1.

~\OneDrive\Υπολογιστής\YEAR 2025-2026\SEMESTER 1\FEEG6042 Introduction to Machine Learning\ML Assignment\ML Assignment - Final Code.py

```python
# -*- coding: utf-8 -*-
"""
Created on Sun Nov 23 14:06:35 2025

"""

"""
Question 1

Base CNN and Overfitting Experiments for MNIST (Even vs Odd)

1) Base model: full training set, designed NOT to overfit

2) EXPERIMENT 1: same CNN, but trained on a REDUCED training set
   -> demonstrates overfitting caused by too little data

3) EXPERIMENT 2: more complex CNN with more parameters, full training set
   -> demonstrates overfitting caused by high model capacity
"""

"Importing all the necessary libraries"

import keras                    # for machine learning models, deep neural networks
import sklearn                  # for preprocessing and one-hot encoding
from matplotlib import pyplot as plt  # for plotting
import time                     # for calculating the computation time
import keras.datasets.mnist as mnist  # for loading the MNIST data set


#%%
"Loading the image data"

# Load MNIST: supervised learning data set of 28x28 grayscale digit images.
# Keras already splits this into a training set and a separate test set so
# we can train on one set and evaluate generalisation on unseen data.

(x_train, y_train), (x_test, y_test) = mnist.load_data(path="mnist.npz")

# x_train contains the training images as 28x28 matrices
# y_train contains the corresponding digit labels (0–9)


#%%
"Familiarisation with the data"

# Quick visual check of the raw data. Looking at some example inputs in the ML workflow
# to make sure nothing is obviously wrong with the data set.

for i in range(10):
    x = x_train[(y_train == i)]
    plt.subplot(2, 5, i + 1)
```

```
    plt.imshow(x[0], cmap='gray')
    plt.axis('off')
plt.suptitle("Example MNIST digits (0–9)")
plt.tight_layout()
plt.show()


#%%
"Preparing the even / odd labels"


# Convert the original 10-way digit labels (0–9) into a binary classification
# problem: even digits → class 0, odd digits → class 1.  This keeps the same
# input images but changes the output space to two classes.

y_train_evod = (y_train % 2).astype('uint8')
y_test_evod = (y_test % 2).astype('uint8')


#%%
"Scaling the images"

import sklearn.preprocessing

# Scale pixel values to [0, 1]. Scaling inputs is an important pre-processing
# step so that the neural network sees inputs within a similar numerical range,
# which helps training and optimisation.

scaler = sklearn.preprocessing.MinMaxScaler(feature_range=(0, 1))

# Convert to float before scaling
x_train = x_train.astype('float32')
x_test = x_test.astype('float32')


# Scale each 28x28 image independently using MinMax scaling.
# For image data we treat each image as a small 2D tensor rather than a flat
# feature vector and scale image-by-image, as discussed in the CNN lab.

def im_scaler(x, scaler):
    # Fit the scaler on a single image and rescale its pixels so the minimum
    # becomes 0 and the maximum becomes 1.  This keeps the image structure but
    # normalises intensity.
    for i, X in enumerate(x[:, :, :]):
        x[i, :, :] = scaler.fit_transform(X.reshape(-1, 1)).reshape(28, 28)
    return x


x_train = im_scaler(x_train, scaler)
x_test = im_scaler(x_test, scaler)


#%%
"Reshaping the images for the CNN"
```

```
106
107 # Reshape data to 4D tensors: (number of samples, height, width, channels).
108 # CNNs expect image tensors where the last dimension indexes the channels
109 # (here 1 channel because MNIST is grayscale).
110 X_train = x_train.reshape(-1, 28, 28, 1)
111 X_test = x_test.reshape(-1, 28, 28, 1)
112
113
114 #%%
115 "Turning the image labels into one-hot-encoding format"
116
117 # One-hot encode the even/odd labels.  For 2 classes this gives vectors
118 # [1, 0] and [0, 1], which match the 2-dimensional softmax output and allow
119 # us to use categorical cross-entropy as the cost/loss function.
120
121 onehot = sklearn.preprocessing.OneHotEncoder()
122 onehot.fit(y_train_evod.reshape(-1, 1))
123
124 Y_train = onehot.transform(y_train_evod.reshape(-1, 1)).toarray()
125 Y_test = onehot.transform(y_test_evod.reshape(-1, 1)).toarray()
126
127
128 #%%
129 "Saving path for the figures"
130
131 # Directory where all convergence plots (accuracy/loss vs epoch) for the three
132 # experiments will be saved so we can compare model performance after training.
133
134 save_path = r"C:\Users\anort\OneDrive\Υπολογιστής\YEAR 2025-2026\SEMESTER 1\FEEG6042
    Introduction to Machine Learning\ML Assignment\plots\PLOTS"
135
136
137 #%%
138 "Function for plotting and saving accuracy and loss"
139
140 # Plot training vs validation accuracy and loss over epochs.
141 # These convergence plots are used to check if the model has converged and
142 # to spot overfitting when training and validation curves diverge.
143
144
145 def plot_and_save(history, title_acc, title_loss):
146
147     # Accuracy plot
148     plt.figure()
149
150     # Accuracy: percentage of correctly classified images for train and test sets
151     # (binary classification accuracy metric).
152     plt.plot(history.history['accuracy'])
153     plt.plot(history.history['val_accuracy'])
154     plt.title(title_acc)
155     plt.xlabel("Epoch")
156     plt.ylabel("Accuracy")
157     plt.legend(['Train', 'Test'], loc='upper left')
158     plt.grid(True, alpha=0.3)
```

```
    plt.savefig(f"{save_path}\\{title_acc}.png", dpi=300, bbox_inches="tight")
    plt.show()

    # Loss plot
    plt.figure()

    # Loss: categorical cross-entropy between true one-hot labels and the
    # softmax output probabilities from the network.
    plt.plot(history.history['loss'])
    plt.plot(history.history['val_loss'])
    plt.title(title_loss)
    plt.xlabel("Epoch")
    plt.ylabel("categorical cross entropy")
    plt.legend(['Train', 'Test'], loc='upper left')
    plt.grid(True, alpha=0.3)
    plt.savefig(f"{save_path}\\{title_loss}.png", dpi=300, bbox_inches="tight")
    plt.show()

#%%
"Function for extracting the final accuracy and loss"


def print_final_metrics(history, model_name):
    final_train_acc = history.history['accuracy'][-1]
    final_val_acc   = history.history['val_accuracy'][-1]

    final_train_loss = history.history['loss'][-1]
    final_val_loss   = history.history['val_loss'][-1]

    print(f"\n{model_name} FINAL METRICS")
    print(f"Training Accuracy: {final_train_acc:.4f}")
    print(f"Validation Accuracy: {final_val_acc:.4f}")
    print(f"Training Loss: {final_train_loss:.4f}")
    print(f"Validation Loss: {final_val_loss:.4f}")


#%%
#-------------------------------------------------------
# BASE MODEL
#-------------------------------------------------------

"Defining the BASE Convolutional Neural Network"

# BASE CNN: base model with few trainable parameters so that it
# should NOT overfit when trained on the full MNIST training set.
# This is the reference model for the overfitting experiments.


cnn_base = keras.models.Sequential()

cnn_base.add(keras.layers.Conv2D(8, kernel_size=(3,3), input_shape=X_train.shape[1:],
activation='relu'))
# First convolutional layer: 8 learnable 3x3 kernels convolve over the input
# image to detect simple local patterns. ReLU adds non-linearity.
```

```
212
213 cnn_base.add(keras.layers.MaxPooling2D(pool_size=(2,2)))
214 # Max pooling: reduces spatial resolution while keeping strongest
215 # activations, lowering dimensionality
216
217 cnn_base.add(keras.layers.Conv2D(16, kernel_size=(3,3), activation='relu'))
218 # Second convolutional layer: 16 filters to learn more complex features
219
220 cnn_base.add(keras.layers.MaxPooling2D(pool_size=(2,2)))
221 # Second pooling layer: further downsampling to reduce feature map size and
222 # number of parameters in the later dense layer.
223
224 cnn_base.add(keras.layers.Flatten())
225 # Flatten 2D feature maps into a 1D feature vector so it can be fed into a
226 # fully connected (dense) output layer.
227
228 cnn_base.add(keras.layers.Dense(2, activation='softmax'))
229 # Dense output layer: 2 output neurons (even vs odd) with softmax activation
230 # to represent predicted class probabilities that sum to 1.
231
232
233 # Use categorical cross-entropy as the loss/cost function for multi-class
234 # classification with softmax outputs and track accuracy as performance
235 # metric. Optimiser 'adam' performs stochastic gradient descent updates.
236 cnn_base.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
237
238 #%%
239 "Training the BASE model"
240
241 # Train the BASE CNN on the full training set. Validation_data=(X_test, Y_test)
242 # keeps the test set  for evaluating generalisation after each epoch.
243
244 start_time = time.time()
245
246 history_base = cnn_base.fit(X_train, Y_train, validation_data=(X_test, Y_test), epochs=20,
    batch_size=64)
247 # 20 epochs: long enough for the base model to converge without
248 # deliberately forcing severe overfitting.
249
250 end_time = time.time()
251 print(f"BASE training time: {end_time - start_time:.2f} seconds")
252
253 # Count the number of trainable parameters.  Model complexity is directly
254 # linked to how many parameters are being learned.
255 print(f"BASE parameters: {cnn_base.count_params()}")
256
257 plot_and_save(history_base, "Base Model - CNN Model accuracy", "Base Model - CNN Model
    Loss")
258
259 print_final_metrics(history_base, "BASE MODEL")
260
261
262 #%%
263 #-----------------------------------------------------
```

```
# EXPERIMENT 1 – LESS TRAINING DATA
#-------------------------------------------------------


# EXPERIMENT 1: keep the same CNN architecture as the base model but train
# it on a much smaller training set (only 1000 images).
# Complex models trained on too little data are prone to overfitting.
subset_size = 1000

# Use only the first 'subset_size' training samples to simulate a limited
# training data scenario.
X_train_small = X_train[:subset_size]
Y_train_small = Y_train[:subset_size]

# Use exactly the same CNN structure as in the base model

cnn_exp1 = keras.models.Sequential()

cnn_exp1.add(keras.layers.Conv2D(8, kernel_size=(3,3),
input_shape=X_train_small.shape[1:], activation='relu'))
cnn_exp1.add(keras.layers.MaxPooling2D(pool_size=(2,2)))

cnn_exp1.add(keras.layers.Conv2D(16, kernel_size=(3,3), activation='relu'))
cnn_exp1.add(keras.layers.MaxPooling2D(pool_size=(2,2)))

cnn_exp1.add(keras.layers.Flatten())
cnn_exp1.add(keras.layers.Dense(2, activation='softmax'))

cnn_exp1.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])

#%%
"Training EXPERIMENT 1"

start_time = time.time()

history_exp1 = cnn_exp1.fit(X_train_small, Y_train_small, validation_data=(X_test,
Y_test), epochs=100, batch_size=64)
# 100 epochs on small dataset: train for much longer

end_time = time.time()
print(f"EXP 1 training time: {end_time - start_time:.2f} seconds")


print(f"EXP 1 parameters: {cnn_exp1.count_params()}")

plot_and_save(history_exp1, "Experiment 1 - CNN Model accuracy", "Experiment 1 - CNN Model
Loss")


print_final_metrics(history_exp1, "EXPERIMENT 1")

#%%
#-------------------------------------------------------
# EXPERIMENT 2 – HIGH MODEL COMPLEXITY
```

```
#-------------------------------------------------------

"Defining a more complex Convolutional Neural Network (EXPERIMENT 2)"

# EXPERIMENT 2: increase model complexity (more filters and a larger dense
# layer) while using the full training set.  This raises the number of
# trainable parameters and should again make overfitting more likely.

cnn_exp2 = keras.models.Sequential()

cnn_exp2.add(keras.layers.Conv2D(32, kernel_size=(3,3), input_shape=X_train.shape[1:],
activation='relu'))
# First conv layer now has 32 filters instead of 8 → higher capacity to fit
# training data.

cnn_exp2.add(keras.layers.MaxPooling2D(pool_size=(2,2)))
# Pooling as before: reduce spatial size and keep strongest activations.

cnn_exp2.add(keras.layers.Conv2D(64, kernel_size=(3,3), activation='relu'))
# Second conv layer with 64 filters increases depth and complexity further.

cnn_exp2.add(keras.layers.MaxPooling2D(pool_size=(2,2)))

cnn_exp2.add(keras.layers.Conv2D(64, kernel_size=(3,3), activation='relu',
padding='same'))
# Extra conv layer with 'same' padding: keeps spatial size while adding more
# feature extraction, again increasing the number of parameters.

cnn_exp2.add(keras.layers.Flatten())
cnn_exp2.add(keras.layers.Dense(64, activation='relu'))
# Additional dense hidden layer with 64 units: fully connected layer that
# allows complex non-linear combinations of extracted features.

cnn_exp2.add(keras.layers.Dense(2, activation='softmax'))

cnn_exp2.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])

#%%
"Training EXPERIMENT 2"

start_time = time.time()

# Train the high-capacity CNN on the full training set.  With many more
# parameters than the base model
history_exp2 = cnn_exp2.fit(X_train, Y_train, validation_data=(X_test, Y_test), epochs=20,
batch_size=64)

end_time = time.time()
print(f"EXP 2 training time: {end_time - start_time:.2f} seconds")


print(f"EXP 2 parameters: {cnn_exp2.count_params()}")

```

```
plot_and_save(history_exp2, "Experiment 2 - CNN Model accuracy", "Experiment 2 - CNN Model Loss")

print_final_metrics(history_exp2, "EXPERIMENT 2")

```